For the Java Persistence API, the `org.hibernate.cacheable` query hint must be provided, so when executing the following query:

```
List<Post> posts = entityManager.createQuery(
    "select p from Post p " +
    "where p.title like :token", Post.class)
.setParameter("token", "High-Performance%")
.setHint("org.hibernate.cacheable", true)
.getResultList();
```

Hibernate stores it using the following cache entry key:

```
key = {QueryKey}
  sqlQueryString = "select p0_.id as id1_0_, p0_.title as title2_0_, p0_.version as \
version3_0_ from Post p0_ where p0_.title like ?"
  positionalParameterTypes = {org.hibernate.type.Type[0]}
  positionalParameterValues = {java.lang.Object[0]}
  namedParameters = {java.util.HashMap}  size = 1
    0 = {java.util.HashMap$Node} "token" -> "High-Performance%"
  firstRow = null
  maxRows = null
  tenantIdentifier = null
  filterKeys = null
  customTransformer = {org.hibernate.transform.CacheableResultTransformer}
  hashCode = -221304300
```

The cache entry value associated with the query above looks like this:

```
element = {net.sf.ehcache.Element}
  key = {org.hibernate.cache.spi.QueryKey}
  value = {java.util.ArrayList}  size = 2
    0 = {java.lang.Long} "5990928755007489"
    1 = {java.lang.Long} "1"
    2 = {java.lang.Long} "2"
  version = 1
  hitCount = 1
  timeToLive = 120
  timeToIdle = 120
  creationTime = 1462629167305
  lastAccessTime = 1462629171227
  lastUpdateTime = 1462629167305
  cacheDefaultLifespan = true
  id = 0
```

The first entry represents the timestamp of the `Session` that stored the given query cache result. When the query cache entry is read, Hibernate checks if the query timestamp is greater than the associated tablespace update timestamps, and it only returns the cached element if there was no update since the cached result was stored.

The second and the third value entries represent the entity identifiers that satisfied these query filtering criteria.

Just like the collection cache, because the query cache only stores entity identifiers, it is mandatory that the associated entities are cached as well.

15.5.6 Cache concurrency strategies

The `usage` property of the `@Cache` annotation specifies the `CacheConcurrencyStrategy` in use for a particular entity or collection. There are four distinct strategies to choose from (`READ_ONLY`, `NONSTRICT_READ_WRITE`, `READ_WRITE`, `TRANSACTIONAL`), each one defining a distinct behavior when it comes to inserting, updating, or deleting entities:

Before starting explaining each particular cache concurrency strategy, it is better to provide some guidelines related to visualizing the cache content. Hibernate can gather statistics about the second-level cache usage, and, as explained in the Hibernate statistics section, the
`hibernate.generate_statistics` configuration property must be set to true.

Once statistics are enabled, it is very easy to inspect the second-level cache regions using the following utility method:

```
protected void printCacheRegionStatistics(String region) {
    SecondLevelCacheStatistics statistics =
        sessionFactory().getStatistics().getSecondLevelCacheStatistics(region);
    LOGGER.debug("\nRegion: {},\nStatistics: {},\nEntries: {}",
        region, statistics, statistics.getEntries());
}
```

As previously explained, enterprise caching requires diligence because data is duplicated between the database, which is also the system of record, and the caching layer. To make sure that the two separate sources of data do not drift apart, Hibernate must synchronize the second-level cache entry whenever the associated entity state is changed. Because it has a great impact on data integrity, as well as on application performance, the following sections will discuss in greater detail each of those cache concurrency strategies.

15.5.6.1 READ_ONLY

If the cached data is immutable, there is no risk of data inconsistencies, so read-only data is always a good candidate for caching.

15.5.6.1.1 Inserting READ_ONLY cache entries

Considering that the previous `Post` entity is using the `READ_ONLY` cache concurrency strategy, when persisting a new entity instance:

```
doInJPA(entityManager -> {
    Post post = new Post();
    post.setId(1L);
    post.setTitle("High-Performance Java Persistence");
    entityManager.persist(post);
});
printCacheRegionStatistics(Post.class.getName());
```

Hibernate generates the following output:

```
INSERT INTO post (title, version, id)
VALUES ('High-Performance Java Persistence', 0, 1)

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=1],
Entries: {1=CacheEntry(Post)[1,High-Performance Java Persistence,0]}
```

The `putCount` value is 1, so the entity is cached on insert, meaning that `READ_ONLY` is a write-through strategy. Afterward, when issuing a direct load operation, Hibernate generates the following cache statistics:

```
doInJPA(entityManager -> {
    Post post = entityManager.find(Post.class, 1L);
    printCacheRegionStatistics(post.getClass().getName());
});


Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=0,putCount=1],
Entries: {1=CacheEntry(Post)[1,High-Performance Java Persistence,0]}
```

The `hitCount` value is 1 because the entity was loaded from the cache, therefore bypassing the database.

For generated identifiers, the write-through entity caching works only for sequences and table generator, so when inserting a `Post` entity that uses the `GenerationType.SEQUENCE` strategy:

```
doInJPA(entityManager -> {
    Post post = new Post();
    post.setTitle("High-Performance Java Persistence");
    entityManager.persist(post);
});
printCacheRegionStatistics(Post.class.getName());
```

Hibernate is going to generate the following output:

```
INSERT INTO post (title, version, id)
VALUES ('High-Performance Java Persistence', 0, 1)

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=1],
Entries: {1=CacheEntry(Post)[1,High-Performance Java Persistence,0]}
```

Unfortunately, for identity columns, `READ_ONLY` uses a read-through cache strategy instead. If the `Post` entity uses the `GenerationType.IDENTITY` strategy, upon inserting the same `Post` entity instance, the second-level cache is not going to store the newly persisted entity:

```
Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=0],
Entries: {}
```

On the other hand, when the entity is fetched for the first time:

```
doInJPA(entityManager -> {
    Post post = entityManager.find(Post.class, 1L);
    printCacheRegionStatistics(post.getClass().getName());
});
```

Hibernate is going to store the entity into the second-level cache:

```
SELECT p.id AS id1_0_0_, p.title AS title2_0_0_, p.version AS version3_0_0_
FROM   post p
WHERE  p.id = 1
```

```
Region: com.vladmihalcea.book.hpjp.hibernate.cache.readonly.Post,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=CacheEntry(Post)[High-Performance Java Persistence,0]}
```

Considering that the `Post` entity has a bidirectional `@OneToMany PostComment` association, and the collection is cached using the `READ_ONLY` strategy, when adding two comments:

```
doInJPA(entityManager -> {
    Post post = entityManager.find(Post.class, 1L);

    PostComment comment1 = new PostComment();
    comment1.setId(1L);
    comment1.setReview("JDBC part review");
    post.addComment(comment1);

    PostComment comment2 = new PostComment();
    comment2.setId(2L);
    comment2.setReview("Hibernate part review");
    post.addComment(comment2);
});
printCacheRegionStatistics(Post.class.getName() + ".comments");
```

Hibernate inserts the two comments in the database, while the collection cache region is not updated:

```
INSERT INTO post_comment (post_id, review, id)
VALUES (1, 'JDBC part review', 1)
INSERT INTO post_comment (post_id, review, id)
VALUES (1, 'Hibernate part review', 2)

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=0],
Entries: {}
```

However, upon requesting the collection for the first time:

```
Post post = entityManager.find(Post.class, 1L);
assertEquals(2, post.getComments().size());
printCacheRegionStatistics(Post.class.getName() + ".comments");
```

Hibernate executes the SQL query and updates the cache as well:

```
SELECT pc.post_id AS post_id3_1_0_, pc.id AS id1_1_0_, pc.review AS review2_1_1_
FROM   post_comment pc
WHERE  pc.post_id = 1

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=CollectionCacheEntry[1,2]}
```

Once the collection is cached, any further collection fetch request is going to be served from the cache, therefore bypassing the database.

As opposed to the `READ_ONLY` entity cache, the `READ_ONLY` collection cache is not write-through. Instead, it uses a read-through caching strategy.

15.5.6.1.2 Updating READ_ONLY cache entries

The `READ_ONLY` strategy disallows updates, so when trying to modify a `Post` entity, Hibernate throws the following exception:

```
java.lang.UnsupportedOperationException: Can't write to a readonly object
```

As of writing (Hibernate 5.1.0), Hibernate allows removing elements from a `READ_ONLY` cached collection. However, it does not invalidate the collection cache entry.

This way, when removing a `PostComment` from the `Post` entity `comments` collection:

```
doInJPA(entityManager -> {
    Post post = entityManager.find(Post.class, 1L);
    PostComment comment = post.getComments().remove(0);
    comment.setPost(null);
});

printCacheRegionStatistics(Post.class.getName());
printCacheRegionStatistics(PostComment.class.getName());

doInJPA(entityManager -> {
    Post post = entityManager.find(Post.class, 1L);
});
```

Hibernate generates the following output:

```
DELETE FROM post_comment WHERE id = 1

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=CollectionCacheEntry[1,2]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=2],
Entries: {2=CacheEntry(PostComment)[1,Hibernate part review]}

javax.persistence.EntityNotFoundException: Unable to find
com.vladmihalcea.book.hpjp.hibernate.cache.PostComment with id 1
```

In reality, every `READ_ONLY` entity and collection should be marked with the `@Immutable` annotation:

```
@Entity @Immutable @Cache(usage = CacheConcurrencyStrategy.READ_ONLY)
public class Post {

    @OneToMany(cascade = CascadeType.PERSIST, mappedBy = "post")
    @Immutable @Cache(usage = CacheConcurrencyStrategy.READ_ONLY)
    private List<PostComment> comments = new ArrayList<>();

    //Code omitted for brevity
}
```

This way, when trying to update a `PostComment` collection, Hibernate is going to throw the following exception:

```
org.hibernate.HibernateException: changed an immutable collection instance:
[com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments#1]
```

15.5.6.1.3 Deleting READ_ONLY cache entries

While updates should never occur for `READ_ONLY` entities (which signals a data access logic issue), deletes are permitted.

When deleting a `Post` entity that happens to be stored in the second-level cache:

```
printCacheRegionStatistics(Post.class.getName());
printCacheRegionStatistics(PostComment.class.getName());

doInJPA(entityManager -> {
    Post post = entityManager.find(Post.class, 1L);
    entityManager.remove(post);
});

printCacheRegionStatistics(Post.class.getName());
printCacheRegionStatistics(PostComment.class.getName());
```

Hibernate generates the following output:

```
Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=2,missCount=0,putCount=1],
Entries: {1=CacheEntry(Post)[1,High-Performance Java Persistence,0]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=2],
Entries: {1=CacheEntry(PostComment)[1,JDBC part review],
          2=CacheEntry(PostComment)[1,Hibernate part review]}

DELETE FROM post_comment WHERE id = 1
DELETE FROM post_comment WHERE id = 2
DELETE FROM post WHERE id = 1 AND version = 0

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=3,missCount=0,putCount=1],
Entries: {}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=2,missCount=0,putCount=2],
Entries: {}
```

The `Post` and `PostComment` entities are successfully removed form the database and the second-level cache as well.

15.5.6.2 NONSTRICT_READ_WRITE

The `NONSTRICT_READ_WRITE` concurrency strategy is designed for entities that are updated infrequently, and when strict consistency is not a mandatory requirement. The following examples are going to reuse the same entities that were previously employed, the only thing being different is that the `Post` and `PostComment` entities, as well as the `comments` collections, are using the `@Cache(usage = CacheConcurrencyStrategy.NONSTRICT_READ_WRITE)` annotation.

15.5.6.2.1 Inserting NONSTRICT_READ_WRITE cache entries

First of all, unlike other strategies, `NONSTRICT_READ_WRITE` is not write-through. Therefore, when persisting a `Post` entity, the second-level cache is not going to store the newly inserted object. Instead, `NONSTRICT_READ_WRITE` is a read-through cache concurrency strategy.

```
doInJPA(entityManager -> {
    Post post = new Post();
    post.setId(1L);
    post.setTitle("High-Performance Java Persistence");

    PostComment comment1 = new PostComment();
    comment1.setId(1L);
    comment1.setReview("JDBC part review");
    post.addComment(comment1);

    PostComment comment2 = new PostComment();
    comment2.setId(2L);
    comment2.setReview("Hibernate part review");
    post.addComment(comment2);

    entityManager.persist(post);
});
printCacheRegionStatistics(Post.class.getName());
printCacheRegionStatistics(Post.class.getName() + ".comments");

LOGGER.info("Load Post entity and comments collection");
doInJPA(entityManager -> {
    Post post = entityManager.find(Post.class, 1L);
    assertEquals(2, post.getComments().size());
    printCacheRegionStatistics(post.getClass().getName());
    printCacheRegionStatistics(Post.class.getName() + ".comments");
});
```

When executing the test case above, the `Post` entity and `PostComment` collections are going to be cached upon being fetched for the first time.

```
INSERT INTO post (title, version, id)
VALUES ('High-Performance Java Persistence', 0, 1)

INSERT INTO post_comment (post_id, review, id) VALUES (1, 'JDBC part review', 1)
INSERT INTO post_comment (post_id, review, id)
VALUES (1, 'Hibernate part review', 2)

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=0],
Entries: {}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=0],
Entries: {}

--Load Post entity and comments collection
SELECT p.id AS id1_0_0_, p.title AS title2_0_0_, p.version AS version3_0_0_
FROM   post p
WHERE  p.id = 1

SELECT pc.post_id AS post_id3_1_0_, pc.id AS id1_1_0_, pc.review AS review2_1_1_
FROM   post_comment pc
WHERE  pc.post_id = 1

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=CacheEntry(Post)[1,High-Performance Java Persistence,0]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=CollectionCacheEntry[1,2]}
```

For it removes cache entries, `NONSTRICT_READ_WRITE` is only appropriate when entities are rarely changed. Otherwise, if the cache miss rate is too high, the cache renders inefficient.

15.5.6.2.2 Updating NONSTRICT_READ_WRITE cache entries

Unlike the READ_ONLY cache concurrency strategy, NONSTRICT_READ_WRITE supports entity and collection modifications.

```
doInJPA(entityManager -> {
    Post post = entityManager.find(Post.class, 1L);
    post.setTitle("High-Performance Hibernate");
    PostComment comment = post.getComments().remove(0);
    comment.setPost(null);
});
printCacheRegionStatistics(Post.class.getName());
printCacheRegionStatistics(Post.class.getName() + ".comments");
printCacheRegionStatistics(PostComment.class.getName());
```

When executing the test case above, Hibernate generates the following output:

```
UPDATE post
SET title = 'High-Performance Hibernate', version = 1
WHERE id = 1 AND version = 0

DELETE FROM post_comment WHERE id = 1

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=1,putCount=1],
Entries: {}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=1,putCount=1],
Entries: {}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=2,missCount=0,putCount=2],
Entries: {2=CacheEntry(PostComment)[1,Hibernate part review]}
```

15.5.6.2.3 Risk of inconsistencies

NONSTRICT_READ_WRITE does not offer strict consistency because it takes no locks on the cache entries that get modified. For this reason, on very tiny time interval, it is possible that the database and the cache might render different results.

During an entity update, the flow of operations goes like this:

1. The current Hibernate transaction (e.g. `JdbcTransaction` or `JtaTransaction`) is flushed.
2. The `DefaultFlushEventListener` executes all pending actions contained in the current `ActionQueue`.
3. The `EntityUpdateAction` calls the `update` method of the `EntityRegionAccessStrategy`.
4. The `NonStrictReadWriteEhcacheCollectionRegionAccessStrategy` removes the cache entry from the underlying 'EhcacheEntityRegion.

After the database transaction is committed, the cache entry is removed once again:

1. The after transaction completion callback is called.
2. The current `Session` propagates this event to its internal `ActionQueue`.
3. The `EntityUpdateAction` calls the `afterUpdate` method on the `EntityRegionAccessStrategy`.
4. The `NonStrictReadWriteEhcacheCollectionRegionAccessStrategy` calls the `remove` method on the underlying `EhcacheEntityRegion`.

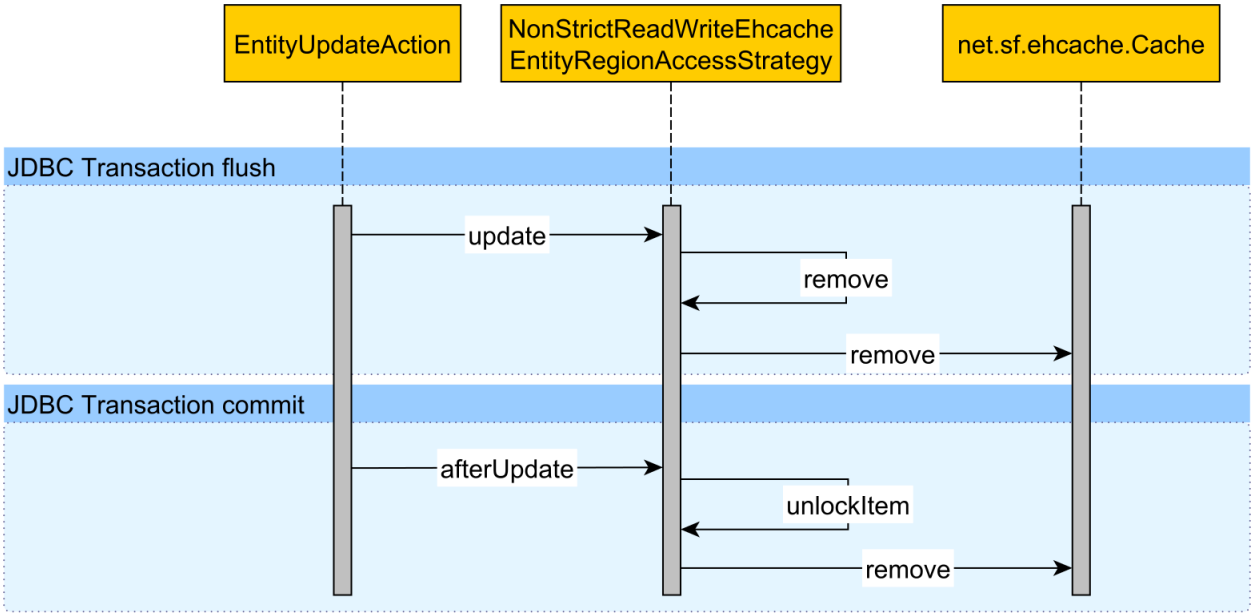

Figure 15.11: NONSTRICT_READ_WRITE update flow

The cache invalidation is not synchronized with the current database transaction. Even if the associated cache region entry gets invalidated twice (before and after transaction completion), there is still a tiny time window when the cache and the database might drift apart.

15.5.6.2.4 Deleting NONSTRICT_READ_WRITE cache entries

When deleting a `Post` entity that cascades the remove event to the `PostComment` collection:

```
printCacheRegionStatistics(Post.class.getName());
printCacheRegionStatistics(Post.class.getName() + ".comments");
printCacheRegionStatistics(PostComment.class.getName());

doInJPA(entityManager -> {
    Post post = entityManager.find(Post.class, 1L);
    entityManager.remove(post);
});

printCacheRegionStatistics(Post.class.getName());
printCacheRegionStatistics(Post.class.getName() + ".comments");
printCacheRegionStatistics(PostComment.class.getName());
```

Hibernate is going to remove all associated cache regions:

```
Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=CacheEntry(Post)[1,High-Performance Java Persistence,0]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=CollectionCacheEntry[1,2]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=2],
Entries: {1=CacheEntry(PostComment)[1,JDBC part review],
          2=CacheEntry(PostComment)[1,Hibernate part review]}
```

```
DELETE FROM post_comment WHERE id = 1
DELETE FROM post_comment WHERE id = 2
DELETE FROM post WHERE id = 1 AND version = 0

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=1,putCount=1],
Entries: {}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=1,putCount=1],
Entries: {}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=2,missCount=0,putCount=2],
Entries: {}
```

Just like with update, the cache entry removal is called twice (the first time during flush and the second time after the transaction is committed).

15.5.6.3 READ_WRITE

To avoid any inconsistency risk while still using a write-through second-level cache, Hibernate offers the `READ_WRITE` cache concurrency strategy. A write-through cache strategy is a much better choice for write-intensive applications since cache entries can be updated rather than being simply removed.

Because the database is the system of record and database operations are wrapped inside one single physical transaction, the cache can either be updated synchronously which requires JTA transactions or asynchronously, right after the database transaction gets committed.

`READ_WRITE` is an asynchronous cache concurrency strategy, and, to prevent data integrity issues like stale cache entries, it employs a soft locking mechanism that provides the guarantees of a logical transaction isolation.

The following examples are going to reuse the same entities that were previously employed, and the only thing that differs is that the `Post` and `PostComment` entities, as well as the `comments` collections, are using the `@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)` annotation.

15.5.6.3.1 Inserting READ_WRITE cache entries

Only the entity cache region can work in write-through mode, and, just like with any other cache concurrency strategy, the collection cache is read-through.

When running the same example used for inserting NONSTRICT_READ_WRITE cache entries, Hibernate generates the following output:

```
INSERT INTO post (title, version, id)
VALUES ('High-Performance Java Persistence', 0, 1)

INSERT INTO post_comment (post_id, review, id)
VALUES (1, 'JDBC part review', 1)
INSERT INTO post_comment (post_id, review, id)
VALUES (1, 'Hibernate part review', 2)

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=1],
Entries: {1=[ value = CacheEntry(Post)[1,High-Performance Java Persistence,0],
              version=0, timestamp=5991931785445376 ]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=0],
Entries: {1=Lock Source-UUID:7d059ff0-0ec8-490f-b316-e77efad0b15f Lock-ID:0}

--Load Post entity and comments collection

SELECT pc.post_id AS post_id3_1_0_, pc.id AS id1_1_0_, pc.review AS review2_1_1_
FROM   post_comment pc
WHERE  pc.post_id = 1

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=0,putCount=1],
Entries: {1=[ value = CacheEntry(Post)[1,High-Performance Java Persistence,0],
              version=0, timestamp=5991931785445376 ]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=[ value = CollectionCacheEntry[1,2],
              version=null, timestamp=5991931785895936 ]}
```

Unfortunately, this write-through caching does not work for the identity columns, and if the Post entity is using the IDENTITY generator:

```
@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

When inserting a `Post` entity:

```
doInJPA(entityManager -> {
    Post post = new Post();
    post.setTitle("High-Performance Java Persistence");
    entityManager.persist(post);
});
printCacheRegionStatistics(Post.class.getName());
```

Hibernate is going to generate the following output:

```
Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=0],
Entries: {}
```

> Because it supports write-through `READ_WRITE` entity caching, the sequence generator is preferred over identity columns. The behavior might change in future, so it is better to check the HHH-7964[a] JIRA issue status.
>
> [a] https://hibernate.atlassian.net/browse/HHH-7964

15.5.6.3.2 Updating READ_WRITE cache entries

As already mentioned, the `READ_WRITE` cache concurrency strategy employs a soft locking mechanism to ensure data integrity.

1. The Hibernate transaction commit procedure triggers a `Session` flush.
2. The `EntityUpdateAction` replaces the current cache entry with a `Lock` object.
3. The `update` method is used for synchronous strategies. Therefore, it is a no-op in this case.
4. The after transaction callbacks are called, and the `EntityUpdateAction` executes the `afterUpdate` method of the `EntityRegionAccessStrategy`.
5. The `ReadWriteEhcacheEntityRegionAccessStrategy` replaces the `Lock` entry with an actual `Item`, encapsulating the entity disassembled state.

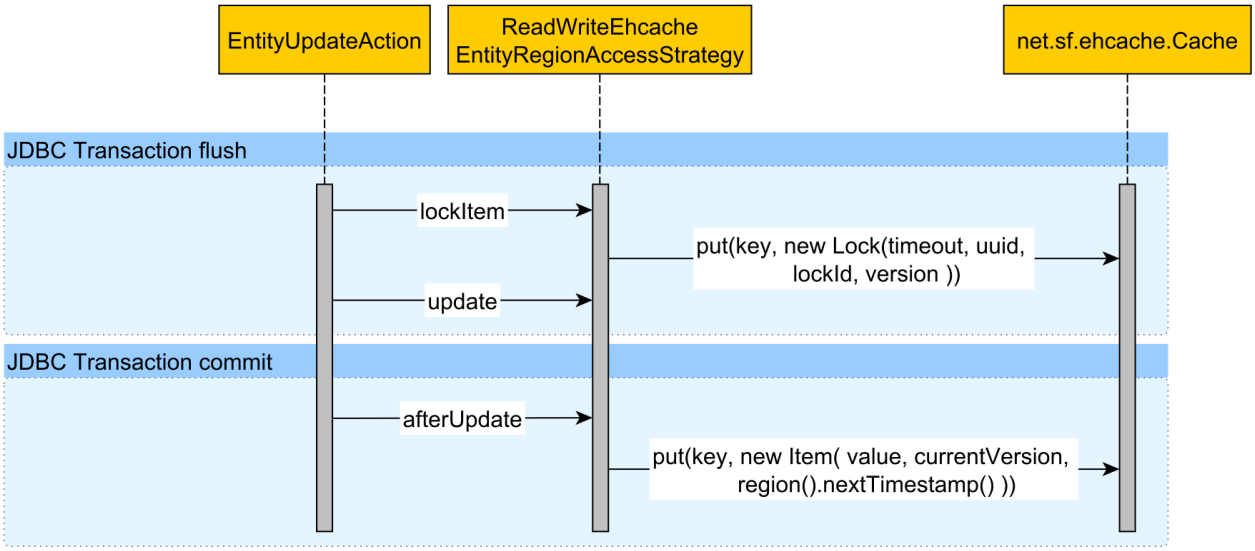

Figure 15.12: READ_WRITE update flow

Just like with database transactions, changes are applied directly, and locks are used to prevent other concurrent transactions from reading uncommitted data. When reading a `Lock` object from the cache, Hibernate knows that the associated entry is being modified by an uncommitted transaction. Therefore, it reads the entity from the database.

To visualize the whole process, when running the following test case:

```
doInJPA(entityManager -> {
    Post post = entityManager.find(Post.class, 1L);
    post.setTitle("High-Performance Hibernate");
    PostComment comment = post.getComments().remove(0);
    comment.setPost(null);
    entityManager.flush();

    printCacheRegionStatistics(Post.class.getName());
    printCacheRegionStatistics(Post.class.getName() + ".comments");
    printCacheRegionStatistics(PostComment.class.getName());
    LOGGER.debug("Commit after flush");
});
printCacheRegionStatistics(Post.class.getName());
printCacheRegionStatistics(Post.class.getName() + ".comments");
printCacheRegionStatistics(PostComment.class.getName());
```

Hibernate generates the following output:

```
UPDATE post
SET title = 'High-Performance Hibernate', version = 1
WHERE id = 1 AND version = 0

DELETE FROM post_comment WHERE id = 1

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=0,putCount=1],
Entries: {1=Lock Source-UUID:69c2fd51-11a3-43c1-9db2-91f30624ac74 Lock-ID:0}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=Lock Source-UUID:e75094c1-6bc2-43f3-87e3-1dcdf6bee083 Lock-ID:1}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=2],
Entries: {1=Lock Source-UUID:99aafdef-7816-43ee-909d-5f10ab759c60 Lock-ID:0,
          2=[ value = CacheEntry(PostComment)[1,Hibernate part review],
              version=null, timestamp=5992022222598145 ]}

--Commit after flush

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=0,putCount=2],
Entries: {1=[ value = CacheEntry(Post)[1,High-Performance Hibernate,1],
              version=1, timestamp=5992019884548096 ]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=Lock Source-UUID:db769a0a-d65a-4911-952e-1d0bb851ed8d Lock-ID:1}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=2],
Entries: {1=Lock Source-UUID:f357da6a-665e-40d1-84c0-760e450df421 Lock-ID:0,
          2=[ value = CacheEntry(PostComment)[1,Hibernate part review],
              version=null, timestamp=5992019884109825 ]}
```

Right after the Persistence Context is flushed, Hibernate executes the associated SQL statements and adds `Lock` objects into the cache entries associated with the currently modifying `Post` entity and `comments` collection, as well as for the deleting `PostComment` entity.

After the transaction is committed, the `Post` entity cache entry is replaced with an `Item` object containing the updated disassembled state. Since `READ_WRITE` collections are not write-through, the `comments` collection cache entry is still a `Lock` object even after commit. Since the `PostComment` entity has been deleted, its cache entry is represented by a `Lock` entry.

15.5.6.3.3 Deleting READ_WRITE cache entries

Deleting entities is similar to the update process, as we can see from the following sequence diagram:

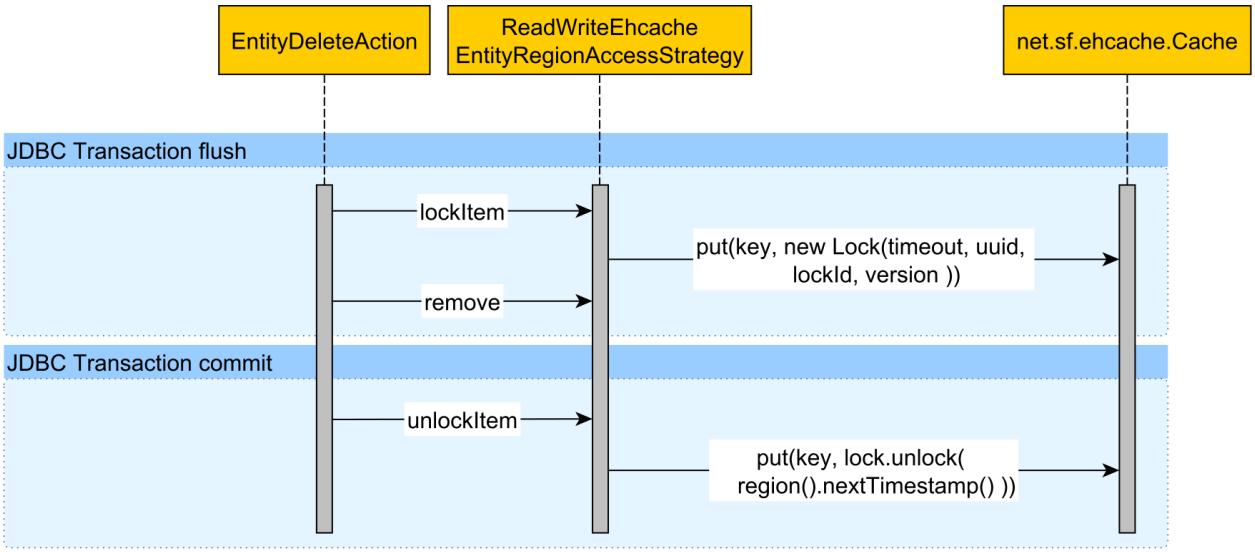

Figure 15.13: READ_WRITE delete flow

1. The Hibernate transaction commit procedure triggers a `Session` flush.
2. The `EntityDeleteAction` replaces the current cache entry with a `Lock` object
3. The `remove` method call doesn't do anything since `READ_WRITE` is an asynchronous cache concurrency strategy.
4. The after transaction callbacks are called, and the `EntityDeleteAction` executes the `unlockItem` method of the `EntityRegionAccessStrategy`.
5. The `ReadWriteEhcacheEntityRegionAccessStrategy` replaces the `Lock` entry with another `Lock` object whose timeout period is further increased.

After an entity is deleted, its associated second-level cache entry will be replaced by a `Lock` object, so that any subsequent request is redirected to reading from the database instead of using the second-level cache entry.

When running the same example used for deleting `NONSTRICT_READ_WRITE` cache entries, Hibernate generates the following output:

```
Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=0,putCount=1],
Entries: {1=[ value = CacheEntry(Post)[1,High-Performance Java Persistence,0],
              version=0, timestamp=5992355751620608 ]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=[ value = CollectionCacheEntry[1,2],
              version=null, timestamp=5992355752042496 ]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=2],
Entries: {1=[ value = CacheEntry(PostComment)[1,JDBC part review],
              version=null, timestamp=5992355751624704 ],
          2=[ value = CacheEntry(PostComment)[1,Hibernate part review],
              version=null, timestamp=5992355751624705 ]}

DELETE FROM post_comment WHERE id = 1
DELETE FROM post_comment WHERE id = 2
DELETE FROM post WHERE id = 1 AND version = 0

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=2,missCount=0,putCount=1],
Entries: {1=Lock Source-UUID:b042192a-9ac6-4877-8663-018f898f1cdb Lock-ID:0}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=1,putCount=1],
Entries: {1=Lock Source-UUID:e75f034a-0346-4696-88fd-1b5100658a6f Lock-ID:1}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=2,missCount=0,putCount=2],
Entries: {1=Lock Source-UUID:1d13b830-e96d-40f5-aa2b-6a402fa6135d Lock-ID:0,
          2=Lock Source-UUID:1d13b830-e96d-40f5-aa2b-6a402fa6135d Lock-ID:1}
```

The delete operation does not remove entries from the second-level cache, but instead it replaces the previous `Item` entries with `Lock` objects. The next time a deleted cache entry is being read, Hibernate is going to redirect the request to the database, therefore guaranteeing strong consistency.

15.5.6.3.4 Soft locking concurrency control

Because the database is the system of record, strong consistency implies that uncommitted cache changes should not be read by other concurrent transactions. The `READ_WRITE` can store either an `Item` or a `Lock`.